

SQL TEMELLERİ

[Kaynak](#)

İÇERİK

1. Veritabanı 101: Temel Kavramlar ve Yapılar
2. SQL (Structured Query Language) ve Veritabanı Yönetimi
3. Veritabanı ve Tablo Komutları (Database & Table Statements)
4. CRUD Operasyonları: Veritabanı Yönetiminin Temeli
5. SQL İfadeleri: SQL Clauses (Sorgu Kriterleri)
6. SQL Operatörleri: Veri Filtreleme ve Karşılaştırma Mantığı
7. SQL Fonksiyonları: Veri Manipülasyonu ve Agregasyon

Veritabanı 101: Temel Kavramlar ve Yapılar

Veritabanları, günümüz dijital dünyasının kalbidir. Instagram'daki beğenilerinizden, Netflix'teki izleme geçmişinize kadar her şey bir veritabanında saklanır. Bir siber güvenlikçi için veritabanlarını anlamak, **SQL Injection** gibi en kritik saldırı türlerini kavramak için ilk adımdır.

Veritabanı Nedir?

Veritabanı, kolayca erişilebilen, yönetilebilen ve analiz edilebilen **organize edilmiş, yapılandırılmış bilgi veya veri koleksiyonudur**. **Kullanım Alanları:** Kimlik Doğrulama: Kullanıcı adı ve şifrelerin saklanması. **Sosyal Medya:** Gönderiler, yorumlar ve beğeniler. **Streaming Servisleri:** İzleme geçmişine göre sunulan öneriler. * **Ticaret:** Stok bilgileri, sipariş geçmişi ve müşteri detayları.

Veritabanı Türleri: SQL vs NoSQL

Veritabanları yapılarına göre iki ana kategoriye ayrılır. Seçim, verinin ne kadar "düzenli" geleceğine bağlıdır.

1. İlişkisel Veritabanları (Relational - SQL)

Verileri belirli bir yapıda (tablolar halinde) saklar. Veri eklenmeden önce yapısı (şema) bellidir.

- **Yapı:** Satırlar ve sütunlar.
- **Örnek:** Bir kullanıcı tablosunda `ad`, `soyad`, `email` alanları sabittir.
- **Kullanım:** Verinin tutarlı ve kesin olması gereken yerler (örn: Bankacılık işlemleri, E-ticaret).

2. İlişkisel Olmayan Veritabanları (Non-Relational - NoSQL)

Verileri tablo formatında değil, belge (document) veya anahtar-değer çifti gibi formatlarda saklar.

- **Yapı:** Esnektir. Her kayıt farklı alanlara sahip olabilir.
- **Örnek JSON Yapısı:**

JSON

```
{
  "_id": ObjectId("4556712cd2b2397ce1b47661"),
  "name": { "first": "Thomas", "last": "Anderson" },
  "occupation": [ "The One"],
  "steps_taken" : 4738947387743977493
}
```

- **Kullanım:** Veri formatının sürekli değişebildiği yerler (örn: Sosyal medya içerikleri, büyük veri analitiği).

Tablolar, Satırlar ve Sütunlar (İlişkisel Yapı)

İlişkisel bir veritabanında her şey bir **Tablo (Table)** içinde saklanır. Örneğin bir kitapçada "Kitaplar" adında bir tablomuz olsun:

- **Sütunlar (Columns):** Tablodaki veri türlerini tanımlar. Örneğin: `id`, `Kitap_Adi`, `Yayin_Tarihi`. Her sütunun bir **Veri Tipi** vardır (String, Integer, Float, Date). Eğer sayı beklenen bir sütuna metin girilirse veritabanı bunu reddeder.
- **Satırlar (Rows):** Tabloya girilen her bir tekil kayıttır.
 - **Örnek:** `1 | Android Security Internals | 2014-10-14` bir satırdır.

Anahtarlar: Veriyi Birbirine Bağlamak

Veritabanındaki farklı tablolar (örneğin "Kitaplar" ve "Yazarlar") arasındaki ilişkiyi **Anahtarlar (Keys)** ile kurarız.

1. Primary Key (Birincil Anahtar)

Bir tablodaki her satırı **benzersiz (unique)** şekilde tanımlayan sütundur.

- **Özellik:** Asla tekrarlanamaz. Bir öğrenci numarası veya T.C. kimlik numarası gibi düşünebiliriz.
- **Örnek:** "Kitaplar" tablosunda `id` sütunu Primary Key'dir. İki farklı kitabın ID'si aynı olamaz.

2. Foreign Key (Yabancı Anahtar)

Bir tablodaki veriyi başka bir tablodaki veriyle ilişkilendirmek için kullanılan sütundur.

- **Mantık:** "Kitaplar" tablosuna bir `yazar_id` sütunu ekleriz. Bu sütun, "Yazarlar" tablosundaki `id` (Primary Key) ile eşleşir.
- **Önem:** İlişkisel veritabanı ismindeki "ilişki" tam olarak buradan gelir. Bir tablodan diğerine köprü kurmamızı sağlar.

Pentester Notu: SQL Injection saldırılarında saldırganlar, `UNION` operatörü gibi tekniklerle bu tablolar arasındaki ilişkileri suistimal ederek yetkisi olmayan tablolardan (örneğin yazarları ararken kullanıcı şifrelerini çekmek gibi) veri sızdırmaya çalışırlar.

SQL (Structured Query Language) ve Veritabanı Yönetimi

Veritabanı teorisini öğrendik; peki bu devasa veri yığınlarını gerçek hayatta nasıl kontrol ediyoruz? İşte burada devreye **SQL** ve **DBMS** (Veritabanı Yönetim Sistemleri) giriyor. Bir siber güvenlik araştırmacısı için SQL bilmek, sadece veri çekmek değil, aynı zamanda veritabanı seviyesindeki zafiyetleri anlamak için (SQLi gibi) zorunludur.

Veritabanı Yönetim Sistemi (DBMS) Nedir?

Bir veritabanı tek başına sadece bir dosyadır. Onunla etkileşime girmek için bir arayüze ihtiyaç duyarız. **DBMS**, son kullanıcı ile veritabanı arasında köprü görevi gören bir yazılımdır.

- **Görevi:** Veriyi çekmek (retrieve), güncellemek (update) ve yönetmek.
- **Popüler Örnekler:** * **MySQL / MariaDB:** Web uygulamalarında en yaygın olanlar.
 - **PostgreSQL:** Gelişmiş özellikleriyle bilinir.
 - **Oracle Database:** Kurumsal ve büyük ölçekli sistemler.
 - **MongoDB:** NoSQL dünyasının liderlerinden.

SQL: Veritabanının Dili

SQL (Yapılandırılmış Sorgu Dili), ilişkisel veritabanlarındaki verileri sorgulamak, tanımlamak ve manipüle etmek için kullanılan standart programlama dilidir.

SQL Neden Bu Kadar Önemli? (Avantajları)

- **Hız:** Devasa veri setleri içinden aradığınız tek bir satırı neredeyse anlık olarak döndürebilir.
- **Öğrenme Kolaylığı:** SQL, düz İngilizceye çok yakın bir yapıdadır (`SELECT name FROM users` gibi). Bu da sözdizimini (syntax) anlamayı kolaylaştırır.
- **Güvenilirlik:** Verilerin belirli bir yapıya (schema) uymasını zorunlu kılar, böylece veri kirliliğini önler.
- **Esneklik:** Karmaşık veri analizi görevlerini tek bir satırda yapmanıza olanak tanır.

Uygulama: MySQL Terminaline Giriş

Teoriyi bırakıp elimizi kirlletme vakti. Bir Linux terminali üzerinden MySQL veritabanına bağlanmak, siber güvenlik laboratuvarlarında en sık yapacağımız işlemlerden biridir.

Bağlantı Komutu ve Parametreleri

Terminali açın ve şu komutu girin:

Bash

```
mysql -u root -p
```

Parametre Açıklamaları:

- `mysql` : MySQL istemcisini çalıştıran temel komut.
- `-u root` : Kullanıcı adını (**user**) belirtir. Burada en yetkili kullanıcı olan **root** ile bağlanıyoruz.
- `-p` : Şifre (**password**) sorulacağını belirtir.

Şifre Girişi

Komutu yazıp `Enter` a bastığınızda sizden şifre isteyecektir:

- Şifre:** `tryhackme` (Not: Terminalde şifre yazarken karakterler görünmez, bu bir güvenlik özelliğidir. Yazıp `Enter`'a basın.)

Başarılı Bağlantı Çıktısı

Bağlantı sağlandığında karşınıza şu tip bir karşılama ekranı gelecektir:

Plaintext

```
Welcome to the MySQL monitor.  Commands end with ; or \g.  
mysql>
```

Kritik Not: MySQL terminalinde yazdığınız her komutun sonuna **noktalı virgül (;)** koymak zorundasınız. Aksi takdirde veritabanı komutun bitmediğini düşünür ve çalıştırmaz.

Veritabanı ve Tablo Komutları (Database & Table Statements)

Artık teoriyi pratiğe dökme vakti. MySQL terminaline bağlandığımıza göre, kendi veritabanımızı ve tablolarımızı nasıl inşa edeceğimizi öğreneceğiz. Bu komutlar, bir sistemin temel yapısını oluştururken kullanılan en temel yapı taşlarıdır.

1. Veritabanı Komutları (Database Statements)

Bir proje için ilk adım, verilerin içinde yaşayacağı bir "ev" yani veritabanı oluşturmaktır.

CREATE DATABASE (Veritabanı Oluşturma)

Yeni bir veritabanı oluşturmak için kullanılır.

- **Komut:** `CREATE DATABASE thm_bookmarket_db;`

Not: Her SQL komutunun sonuna `;` koymayı unutmayın.

SHOW DATABASES (Veritabanlarını Listeleme)

Sistemde mevcut olan tüm veritabanlarını görmenizi sağlar.

- **Komut:** `SHOW DATABASES;`
- **Çıktı:** Kendi oluşturduğunuzun yanı sıra `mysql`, `information_schema` (veritabanı meta verilerini tutar), `performance_schema` ve `sys` gibi MySQL'in çalışması için gerekli varsayılan tabloları da görürsünüz.

USE (Veritabanını Seçme)

Birden fazla veritabanı olabilir. Hangi veritabanı üzerinde işlem yapacağımızı sisteme söylememiz gerekir.

- **Komut:** `USE thm_bookmarket_db;`
- **Anlamı:** "Şu andan itibaren yazacağım tüm tablo komutlarını bu veritabanı içinde çalıştır."

DROP DATABASE (Veritabanını Silme)

Artık ihtiyaç duyulmayan bir veritabanını tüm içeriğiyle birlikte kalıcı olarak siler. **Geri dönüşü yoktur!**

- **Komut:** `DROP DATABASE database_name;`

2. Tablo Komutları (Table Statements)

Veritabanını seçtikten sonra, verileri organize etmek için tablolar oluştururuz.

CREATE TABLE (Tablo Oluşturma)

Tablo oluştururken kolon isimlerini ve bu kolonların ne tür veri tutacağını (Data Type) belirtmek zorundayız.

Uygulama: Kitap envanteri için bir tablo oluşturalım:

SQL

```
CREATE TABLE book_inventory (  
    book_id INT AUTO_INCREMENT PRIMARY KEY,  
    book_name VARCHAR(255) NOT NULL,  
    publication_date DATE  
);
```

Parametre Detayları:

- **book_id INT:** Bu kolon tam sayı (Integer) tutar.
- **AUTO_INCREMENT:** İlk kayıt 1, ikinci kayıt 2 olacak şekilde ID'yi otomatik artırır.
- **PRIMARY KEY:** Bu kolonu tablonun benzersiz kimliği yapar (TC No gibi).
- **VARCHAR(255):** Değişken karakterli metin tutar, maksimum 255 karakter uzunluğundadır.
- **NOT NULL:** Bu alan boş bırakılamaz; kitap adı girilmeden kayıt yapılamaz.
- **DATE:** Yıl-Ay-Gün formatında tarih tutar.

SHOW TABLES & DESCRIBE (Tabloyu İnceleme)

- **SHOW TABLES;** : Mevcut veritabanındaki tabloları listeler.
- **DESCRIBE book_inventory;** : Tablonun yapısını (hangi kolonlar var, tipleri ne, hangisi anahtar) detaylıca gösterir.

ALTER TABLE (Tabloyu Değiştirme)

Tabloyu oluşturduktan sonra yeni bir kolon eklemek veya bir kolonu değiştirmek isterseniz kullanılır. Örneğin, sayfa sayısı kolonu ekleyelim:

SQL

```
ALTER TABLE book_inventory  
ADD page_count INT;
```

- **Kullanım Alanı:** Kolon adını değiştirme, veri tipini güncelleme veya bir kolonu silme işlemlerinde de kullanılır.

DROP TABLE (Tabloyu Silme)

Belirli bir tabloyu veritabanından tamamen siler.

- **Komut:** `DROP TABLE table_name;`

Özet Akış Şeması

1. `CREATE DATABASE ...` (Evi yap)
2. `USE ...` (Evin içine gir)

3. `CREATE TABLE ...` (Odayı/Mobilyayı kur)
4. `DESCRIBE ...` (Kurduğun yapıyı kontrol et)

CRUD Operasyonları: Veritabanı Yönetiminin Temeli

Veritabanı dünyasında her şey dört temel işlem etrafında döner: **CRUD**. Bu kısaltma; **C**reate (Oluştur), **R**ead (Oku), **U**ppdate (Güncelle) ve **D**eleate (Sil) kelimelerinin baş harflerinden oluşur. Bu operasyonlar, herhangi bir veri yönetim sisteminin olmazsa olmazıdır.

Bu görevde `thm_books` veritabanındaki `books` tablosunu kullanacağız. Başlamadan önce şu komutu çalıştırmayı unutma:

```
USE thm_books;
```

1. Create Operasyonu (INSERT INTO)

Create işlemi, bir tabloya yeni kayıtlar eklemek için kullanılır. SQL'de bu işlem `INSERT INTO` komutu ile gerçekleştirilir.

Kullanım ve Sözdizimi:

SQL

```
INSERT INTO books (id, name, published_date, description)
VALUES (1, "Android Security Internals", "2014-10-14", "An In-Depth Guide to
Android's Security Architecture");
```

Detaylı Analiz:

- `INSERT INTO books (...)` : Verinin hangi tabloya ve hangi kolonlara (sütunlara) ekleneceğini belirtir.
- `VALUES (...)` : Belirtilen kolonlara sırasıyla hangi değerlerin atanacağını söyler.
 - `id` : 1
 - `name` : "Android Security Internals"
 - `published_date` : "2014-10-14"
 - `description` : Kitabın açıklaması.

2. Read Operasyonu (SELECT)

Read işlemi, tablodaki verileri görüntülemek veya geri çağırmak için kullanılır. En temel SQL komutu olan `SELECT` burada devreye girer.

Tüm Sütunları Okuma:

Eğer tablodaki her şeyi görmek istiyorsak yıldız (`*`) sembolünü kullanırız:

SQL

```
SELECT * FROM books;
```

- *: "Tüm sütunları getir" anlamına gelir.
- FROM books : Hangi tablodan veri çekileceğini belirtir.

```
mysql> SELECT * FROM books;
+-----+-----+-----+-----+
| id | name                | published_date | description                                     |
+-----+-----+-----+-----+
| 1 | Android Security Internals | 2014-10-14    | An In-Depth Guide to Android's Security Architecture |
+-----+-----+-----+-----+

1 row in set (0.00 sec)
```

Belirli Sütunları Okuma:

Sadece kitap adını ve açıklamasını görmek istiyorsak kolon isimlerini açıkça yazarız:

SQL

```
SELECT name, description FROM books;
```

- Bu yöntem, büyük veritabanlarında performansı artırır ve sadece ihtiyacımız olan veriye odaklanmamızı sağlar.

```
mysql> SELECT name, description FROM books;
+-----+-----+
| name                | description                                     |
+-----+-----+
| Android Security Internals | An In-Depth Guide to Android's Security Architecture |
+-----+-----+

1 row in set (0.00 sec)
```

3. Update Operasyonu (UPDATE)

Update işlemi, tabloda halihazırda var olan bir kaydı değiştirmek için kullanılır.

Kullanım ve Kritik Uyarı:

SQL

```
UPDATE books
SET description = "An In-Depth Guide to Android's Security Architecture."
WHERE id = 1;
```

Parametre Açıklamaları:

- UPDATE books : Güncellenecek tabloyu seçer.

- `SET description = "..."` : Hangi sütunun yeni değerinin ne olacağını belirler.
- `WHERE id = 1` : **Çok Kritik!** Eğer `WHERE` koşulunu yazmazsanız, tablodaki **tüm** kitapların açıklaması aynı olur. `WHERE` ile sadece `id` değeri 1 olan satırı hedefliyoruz.

```
mysql> UPDATE books
  SET description = "An In-Depth Guide to Android's Security Architecture."
  WHERE id = 1;

Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

4. Delete Operasyonu (DELETE)

Delete işlemi, tablodan belirli bir kaydı tamamen kaldırmak için kullanılır.

Kullanım:

SQL

```
DELETE FROM books WHERE id = 1;
```

- `DELETE FROM books` : Verinin silineceği tabloyu belirtir.
- `WHERE id = 1` : Hangi kaydın silineceğini söyler. `WHERE` kullanılmazsa tablodaki tüm satırlar silinir (tablo boş kalır).

```
mysql> DELETE FROM books WHERE id = 1;

Query OK, 1 row affected (0.00 sec)
```

CRUD Özeti ve Komut Eşleşmeleri

Operasyon	SQL Komutu	İşlev
Create	INSERT	Tabloya yeni bir satır/kayıt ekler.
Read	SELECT	Tablodaki verileri sorgular ve getirir.
Update	UPDATE	Mevcut bir veriyi günceller.
Delete	DELETE	Bir kaydı kalıcı olarak siler.

Pentester İpucu: Siber güvenlikte, bir saldırgan `UPDATE` veya `DELETE` komutlarındaki `WHERE` koşulunu manipüle ederek (`1=1` gibi ifadelerle) tüm veritabanını bozabilir veya silebilir. Bu yüzden kullanıcıdan gelen verilerin doğrudan sorguya dahil edilmesi (SQL Injection) çok tehlikelidir.

SQL İfadeleri: SQL Clauses (Sorgu Kriterleri)

SQL'de **Clause** (İfade/Cümlecik), verileri manipüle ederken veya çekerken kullanılan kriterleri belirleyen parçalardır. Daha önce gördüğümüz `FROM` (tablo seçimi) ve `WHERE` (satır filtreleme) temel ifadelerdir. Bu bölümde ise veriyi tekilleştirme, gruptama ve sıralama gibi ileri düzey kontrol sağlayan yapıları inceleyeceğiz.

Örneklerimizde `thm_books` veritabanındaki `books` tablosuyla devam ediyoruz.

Hazırlık: `USE thm_books;`

1. DISTINCT İfadesi (Tekilleştirme)

`DISTINCT` ifadesi, bir sorgu sonucunda tekrar eden (duplicate) kayıtları engellemek ve sadece **benzersiz (unique)** değerleri döndürmek için kullanılır.

- **Senaryo:** `books` tablosunda "Ethical Hacking" isimli iki farklı kayıt (farklı ID veya açıklamalarla) olabilir.
- **Kullanım:**

"Pasted image 20260329195534.png" bulunamadı.

SQL

```
SELECT DISTINCT name FROM books;
```

- **Sonuç:** Normal bir `SELECT` sorgusu 6 satır döndürürken (2 tane Ethical Hacking dahil), `DISTINCT` kullandığımızda sadece 5 satır döner. Aynı isimli kitaplar tek bir satıra indirgenir.

```
Terminal

mysql> SELECT DISTINCT name FROM books;
+-----+
| name                |
+-----+
| Android Security Internals |
| Bug Bounty Bootcamp      |
| Car Hacker's Handbook    |
| Designing Secure Software |
| Ethical Hacking          |
+-----+

5 rows in set (0.00 sec)
```

2. GROUP BY İfadesi (Gruplama)

`GROUP BY`, verileri belirli bir sütuna göre gruplandırır. Genellikle **toplama/sayma (aggregate)** fonksiyonlarıyla (`COUNT`, `SUM`, `AVG`) birlikte kullanılır.

- **Örnek:** Hangi kitaptan kaç tane olduğunu görmek isteyelim:

SQL

```
SELECT name, COUNT(*)
FROM books
GROUP BY name;
```

- **Analiz:** Veritabanı isimleri aynı olan satırları bir araya getirir. "Ethical Hacking" için COUNT(*) (satır sayısı) sonucu **2** olarak görünürken, diğerleri için **1** görünür. Bu, veritabanındaki veri dağılımını anlamak için kritiktir.

```
mysql> SELECT name, COUNT(*)
      FROM books
      GROUP BY name;
+-----+-----+
| name                | COUNT(*) |
+-----+-----+
| Android Security Internals |      1 |
| Bug Bounty Bootcamp      |      1 |
| Car Hacker's Handbook    |      1 |
| Designing Secure Software |      1 |
| Ethical Hacking           |      2 |
+-----+-----+

5 rows in set (0.00 sec)
```

3. ORDER BY İfadesi (Sıralama)

Sorgu sonuçlarını belirli bir sütuna göre küçükten büyüğe veya büyükten küçüğe sıralamak için kullanılır.

ASC (Ascending - Artan Sıralama)

Verileri en düşükten (veya en eskiden) başlayarak sıralar. ORDER BY varsayılan olarak bu şekilde çalışır.

SQL

```
SELECT * FROM books
ORDER BY published_date ASC;
```

- **Sonuç:** Önce 2014 yılında yayınlanan kitap, en son 2021 sonunda yayınlanan kitap görünür.

```
ASCENDING ORDER
mysql> SELECT *
  FROM books
 ORDER BY published_date ASC;
+-----+-----+-----+-----+
| id | name                | published_date | description                                     |
+-----+-----+-----+-----+
| 1 | Android Security Internals | 2014-10-14    | An In-Depth Guide to Android's Security Architecture |
| 3 | Car Hacker's Handbook    | 2016-02-25    | A Guide for the Penetration Tester                 |
| 5 | Ethical Hacking          | 2021-11-02    | A Hands-on Introduction to Breaking In              |
| 6 | Ethical Hacking          | 2021-11-02    |                                                       |
| 2 | Bug Bounty Bootcamp      | 2021-11-16    | The Guide to Finding and Reporting Web Vulnerabilities |
| 4 | Designing Secure Software | 2021-12-21    | A Guide for Developers                             |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

DESC (Descending - Azalan Sıralama)

Verileri en yüksekten (veya en yeniden) başlayarak sıralar.

SQL

```
SELECT * FROM books
ORDER BY published_date DESC;
```

- **Sonuç:** Listenin en başında en yeni kitaplar yer alır.

```
DESCENDING ORDER
mysql> SELECT *
  FROM books
 ORDER BY published_date DESC;
+-----+-----+-----+-----+
| id | name                | published_date | description                                     |
+-----+-----+-----+-----+
| 4 | Designing Secure Software | 2021-12-21    | A Guide for Developers                             |
| 2 | Bug Bounty Bootcamp      | 2021-11-16    | The Guide to Finding and Reporting Web Vulnerabilities |
| 5 | Ethical Hacking          | 2021-11-02    | A Hands-on Introduction to Breaking In              |
| 6 | Ethical Hacking          | 2021-11-02    |                                                       |
| 3 | Car Hacker's Handbook    | 2016-02-25    | A Guide for the Penetration Tester                 |
| 1 | Android Security Internals | 2014-10-14    | An In-Depth Guide to Android's Security Architecture |
+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

4. HAVING İfadesi (Gruplanmış Veri Filtreleme)

HAVING ifadesi, WHERE ifadesine çok benzer ancak devasa bir farkı vardır: WHERE ham satırları filtrelerken, HAVING **gruplandırılmış sonuçları filtreler**.

- **Senaryo:** Önce kitapları isimlerine göre gruplayalım ve sayalım, sonra sadece isminin içinde "Hack" geçen grupları getirelim.
- **Kullanım:**

SQL

```
SELECT name, COUNT(*)
FROM books
GROUP BY name
HAVING name LIKE '%Hack%';
```

- **Neden WHERE Değil?:** Eğer toplama fonksiyonu (COUNT gibi) sonucuna göre bir filtreleme yapacaksanız (örneğin: `HAVING COUNT(*) > 1`), WHERE kullanamazsınız; mutlaka `HAVING` kullanmanız gerekir.
- **LIKE '%Hack%':** İsmi başında veya sonunda ne olursa olsun içinde "Hack" geçenleri bulur.

```
mysql> SELECT name, COUNT(*)
FROM books
GROUP BY name
HAVING name LIKE '%Hack%';
+-----+-----+
| name                | COUNT(*) |
+-----+-----+
| Car Hacker's Handbook |         1 |
| Ethical Hacking      |         2 |
+-----+-----+
2 rows in set (0.00 sec)
```

Özet Karşılaştırma Tablosu

Clause	Görevi	Ne Zaman Kullanılır?
DISTINCT	Tekrarları siler.	Benzersiz bir liste (örn: şehir listesi) çekerken.
GROUP BY	Satırları gruplar.	İstatistik çıkarırken (sayma, toplama).
ORDER BY	Sonucu sıralar.	Verileri tarihe, fiyata veya isme göre dizerken.
HAVING	Grupları filtreler.	Gruplama yapıldıktan sonra kriter koyarken.

SQL Operatörleri: Veri Filtreleme ve Karşılaştırma Mantığı

SQL operatörleri, veritabanından veri çekerken kullandığımız "filtrelerdir". Mantıksal ve karşılaştırma operatörlerini anlamak, milyonlarca satır arasından sadece ihtiyacımız olan spesifik bilgiyi süzüp almamızı sağlar.

Bu notlarda `thm_books2` veritabanındaki `books` tablosunu kullanacağız.

Hazırlık: `USE thm_books2;`

1. Mantıksal Operatörler (Logical Operators)

Bu operatörler bir koşulun doğruluğunu test eder ve sonuç olarak `TRUE` (doğru) veya `FALSE` (yanlış) döndürür.

LIKE Operatörü

Belirli bir sütun içinde özel bir **kalıbı (pattern)** aramak için kullanılır. Genellikle % (joker karakter) ile birlikte kullanılır.

- % : Herhangi bir sayıda karakteri temsil eder.
- **Örnek:** Açıklamasında "guide" kelimesi geçen kitapları getir.

SQL

```
SELECT * FROM books WHERE description LIKE "%guide%";
```

```
mysql> SELECT *
FROM books
WHERE description LIKE "%guide%";
```

id	name	published_date	description	category
1	Android Security Internals	2014-10-14	An In-Depth Guide to Android's Security Architecture	Defensive Security
2	Bug Bounty Bootcamp	2021-11-16	The Guide to Finding and Reporting Web Vulnerabilities	Offensive Security
3	Car Hacker's Handbook	2016-02-25	A Guide for the Penetration Tester	Offensive Security
4	Designing Secure Software	2021-12-21	A Guide for Developers	Defensive Security

```
4 rows in set (0.00 sec)
```

AND Operatörü

Birden fazla koşulun **aynı anda** doğru olmasını bekler.

- **Örnek:** Kategorisi "Offensive Security" **ve** adı "Bug Bounty Bootcamp" olan kitabı getir.

SQL

```
SELECT * FROM books WHERE category = "Offensive Security" AND name = "Bug Bounty Bootcamp";
```

```
mysql> SELECT *
FROM books
WHERE category = "Offensive Security" AND name = "Bug Bounty Bootcamp";
```

id	name	published_date	description	category
2	Bug Bounty Bootcamp	2021-11-16	The Guide to Finding and Reporting Web Vulnerabilities	Offensive Security

```
1 row in set (0.00 sec)
```

OR Operatörü

Koşullardan **en az birinin** doğru olması durumunda kayıtları getirir.

- **Örnek:** Adında "Android" **veya** "iOS" geçen kitapları listele.

SQL

```
SELECT * FROM books WHERE name LIKE "%Android%" OR name LIKE "%iOS%";
```

```
mysql> SELECT *
FROM books
WHERE name LIKE "%Android%" OR name LIKE "%iOS%";
+-----+-----+-----+-----+-----+
| id | name | published_date | description | category |
+-----+-----+-----+-----+-----+
| 1 | Android Security Internals | 2014-10-14 | An In-Depth Guide to Android's Security Architecture | Defensive Security |
+-----+-----+-----+-----+-----+

1 row in set (0.00 sec)
```

NOT Operatörü

Belirtilen koşulun tam tersini alır, yani o koşula uymayanları getirir.

- **Örnek:** Açıklamasında "guide" kelimesi **geçmeyen** kitapları getir.

SQL

```
SELECT * FROM books WHERE NOT description LIKE "%guide%";
```

```
mysql> SELECT *
FROM books
WHERE NOT description LIKE "%guide%";
+-----+-----+-----+-----+-----+
| id | name | published_date | description | category |
+-----+-----+-----+-----+-----+
| 5 | Ethical Hacking | 2021-11-02 | A Hands-on Introduction to Breaking In | Offensive Security |
+-----+-----+-----+-----+-----+

1 row in set (0.00 sec)
```

BETWEEN Operatörü

Verinin belirli bir **aralıkta** olup olmadığını kontrol eder (başlangıç ve bitiş değerleri dahildir).

- **Örnek:** ID değeri 2 ile 4 arasında olanları getir.

SQL

```
SELECT * FROM books WHERE id BETWEEN 2 AND 4;
```

```
mysql> SELECT *
  FROM books
 WHERE id BETWEEN 2 AND 4;
+----+-----+-----+-----+-----+
| id | name                | published_date | description                                     | category          |
+----+-----+-----+-----+-----+
| 2  | Bug Bounty Bootcamp | 2021-11-16    | The Guide to Finding and Reporting Web Vulnerabilities | Offensive Security |
| 3  | Car Hacker's Handbook | 2016-02-25    | A Guide for the Penetration Tester                 | Offensive Security |
| 4  | Designing Secure Software | 2021-12-21    | A Guide for Developers                             | Defensive Security |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. Karşılaştırma Operatörleri (Comparison Operators)

Değerleri birbirleriyle kıyaslamak için kullanılır.

Eşitlik Operatörleri

- WHERE name = "Designing Secure Software" (Sadece bu isimdekini getirir).

```
mysql> SELECT *
  FROM books
 WHERE name = "Designing Secure Software";
+----+-----+-----+-----+-----+
| id | name                | published_date | description                                     | category          |
+----+-----+-----+-----+-----+
| 4  | Designing Secure Software | 2021-12-21    | A Guide for Developers                             | Defensive Security |
+----+-----+-----+-----+-----+
1 row in set (0.10 sec)
```

- != (Eşit Değildir): Belirtilen değer dışındaki her şeyi getirir.
 - WHERE category != "Offensive Security" (Offensive Security dışındaki kategorileri getirir).

```
mysql> SELECT *
  FROM books
 WHERE category != "Offensive Security";
+----+-----+-----+-----+-----+
| id | name                | published_date | description                                     | category          |
+----+-----+-----+-----+-----+
| 1  | Android Security Internals | 2014-10-14    | An In-Depth Guide to Android's Security Architecture | Defensive Security |
| 4  | Designing Secure Software | 2021-12-21    | A Guide for Developers                             | Defensive Security |
+----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Büyüklik ve Küçüklük Operatörleri

Sadece sayılar için değil, tarihler ve alfabetik sıralamalar için de kullanılır.

Operatör	Anlamı	Örnek (WHERE published_date ...)
<	Küçüktür	< "2020-01-01" (2020'den önce yayınlananlar)

Operatör	Anlamı	Örnek (WHERE published_date ...)
>	Büyüktür	> "2020-01-01" (2020'den sonra yayınlananlar)
<=	Küçük Eşittir	<= "2021-11-15" (15 Kasım 2021 ve öncesi)
>=	Büyük Eşittir	>= "2021-11-02" (2 Kasım 2021 ve sonrası)

```
mysql> SELECT *
FROM books
WHERE published_date > "2020-01-01";
+-----+-----+-----+-----+-----+
| id | name                | published_date | description                                | category          |
+-----+-----+-----+-----+-----+
| 2 | Bug Bounty Bootcamp | 2021-11-16    | The Guide to Finding and Reporting Web Vulnerabilities | Offensive Security |
| 4 | Designing Secure Software | 2021-12-21    | A Guide for Developers                                | Defensive Security |
| 5 | Ethical Hacking      | 2021-11-02    | A Hands-on Introduction to Breaking In                | Offensive Security |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Önemli Notlar ve İpuçları

- Metin vs Sayı:** Metin (String) ve Tarih (Date) değerlerini sorgularken mutlaka tırnak işareti (" . . . " veya ' . . . ') kullanmalısınız. Sayılarda tırnağa gerek yoktur.
- Operatör Önceliği:** Karmaşık sorgularda AND operatörü OR operatöründen önce işlenir. Bu karışıklığı önlemek için parantez kullanmak en güvenli yoldur: WHERE (name LIKE "%A%" OR name LIKE "%B%") AND id > 5 .
- Siber Güvenlik Açısından:** SQL operatörleri, **Blind SQL Injection** saldırılarında saldırganın veritabanına "Evet/Hayır" soruları sormasını sağlar. Örneğin; AND (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin') = 'a' gibi bir sorguyla şifrenin ilk harfinin 'a' olup olmadığını test edebilirler.

SQL Fonksiyonları: Veri Manipülasyonu ve Agregasyon

SQL'de fonksiyonlar, verileri çekerken anlık olarak işlemek, metinleri birleştirmek veya sayısal hesaplamalar yapmak için kullandığımız araçlardır. Bunlar sorgularımızı çok daha güçlü ve okunabilir hale getirir.

Siber güvenlik perspektifinden bakıldığında; özellikle **String** fonksiyonları, veritabanından parça parça veri sızdırmak (örneğin SUBSTRING ile şifrenin karakterlerini tek tek kontrol etmek) için sıkça kullanılır.

1. Metin (String) Fonksiyonları

Metinsel veriler üzerinde işlem yaparak yeni değerler döndürürler.

CONCAT() - Metin Birleştirme

İki veya daha fazla sütunu/metni tek bir sütunda birleştirir.

SQL

```
SELECT CONCAT(name, " is a type of ", category, " book.") AS book_info FROM books;
```

- **Kullanımı:** Farklı kolonlardaki verileri anlamlı bir cümle haline getirmek için idealdir. AS book_info ifadesi ile bu birleşmiş sonuca geçici bir isim (alias) veriyoruz.

```
Terminal

mysql> SELECT CONCAT(name, " is a type of ", category, " book.") AS book_info FROM books;
+-----+
| book_info |
+-----+
| Android Security Internals is a type of Defensive Security book. |
| Bug Bounty Bootcamp is a type of Offensive Security book. |
| Car Hacker's Handbook is a type of Offensive Security book. |
| Designing Secure Software is a type of Defensive Security book. |
| Ethical Hacking is a type of Offensive Security book. |
+-----+

5 rows in set (0.00 sec)
```

GROUP_CONCAT() - Satırları Birleştirme

Birden fazla satırdaki veriyi, aralarına ayırıcı (separator) koyarak tek bir alana toplar.

SQL

```
SELECT category, GROUP_CONCAT(name SEPARATOR ", ") AS books
FROM books
GROUP BY category;
```

- **Siber Güvenlik Notu:** Veritabanındaki tüm tablo isimlerini veya kolon isimlerini tek bir satırda çekmek için saldırganların en sevdiği fonksiyonlardan biridir.

```
Terminal

mysql> SELECT category, GROUP_CONCAT(name SEPARATOR ", ") AS books
FROM books
GROUP BY category;
+-----+
| category | books |
+-----+
| Defensive Security | Android Security Internals, Designing Secure Software |
| Offensive Security | Bug Bounty Bootcamp, Car Hacker's Handbook, Ethical Hacking |
+-----+

2 rows in set (0.01 sec)
```

SUBSTRING() - Parça Alma

Bir metnin belirli bir konumundan başlayarak belirli bir uzunlukta parça alır.

- **Sözdizimi:** SUBSTRING(kolon, baslangic_pozisyonu, uzunluk)

SQL

```
SELECT SUBSTRING(published_date, 1, 4) AS published_year FROM books;
```

- **Örnek:** "2014-10-14" değerinin 1. karakterinden başlayıp 4 karakter olarak sadece "2014" (yıl) bilgisini çeker.

```
mysql> SELECT SUBSTRING(published_date, 1, 4) AS published_year FROM books;
+-----+
| published_year |
+-----+
| 2014          |
| 2021          |
| 2016          |
| 2021          |
| 2021          |
+-----+

5 rows in set (0.00 sec)
```

LENGTH() - Uzunluk Hesaplama

Bir metindeki karakter sayısını (boşluklar dahil) döndürür.

SQL

```
SELECT LENGTH(name) AS name_length FROM books;
```

```
mysql> SELECT LENGTH(name) AS name_length FROM books;
+-----+
| name_length |
+-----+
|          26 |
|          19 |
|          21 |
|          25 |
|          15 |
+-----+

5 rows in set (0.00 sec)
```

2. Toplama (Aggregate) Fonksiyonları

Birden fazla satırdaki değerleri işleyerek tek bir sonuç üretirler.

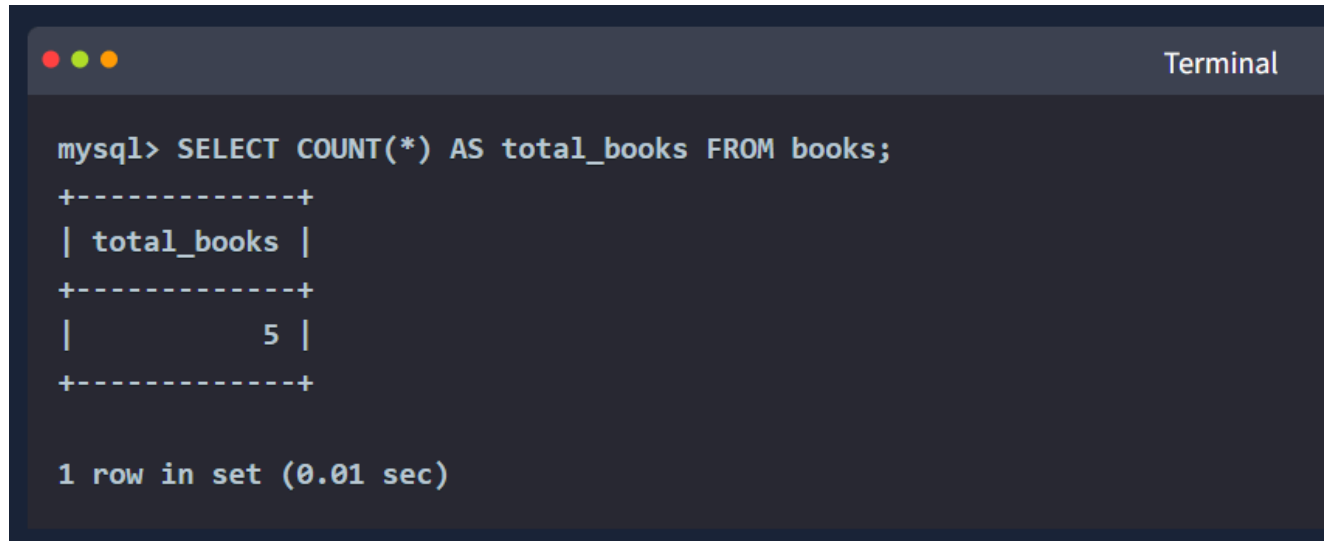
COUNT() - Sayma

Bir tablodaki toplam kayıt (satır) sayısını döndürür.

SQL

```
SELECT COUNT(*) AS total_books FROM books;
```

- `COUNT(*)` : Boş olsun olmasın tüm satırları sayar.



```
mysql> SELECT COUNT(*) AS total_books FROM books;
+-----+
| total_books |
+-----+
|          5 |
+-----+

1 row in set (0.01 sec)
```

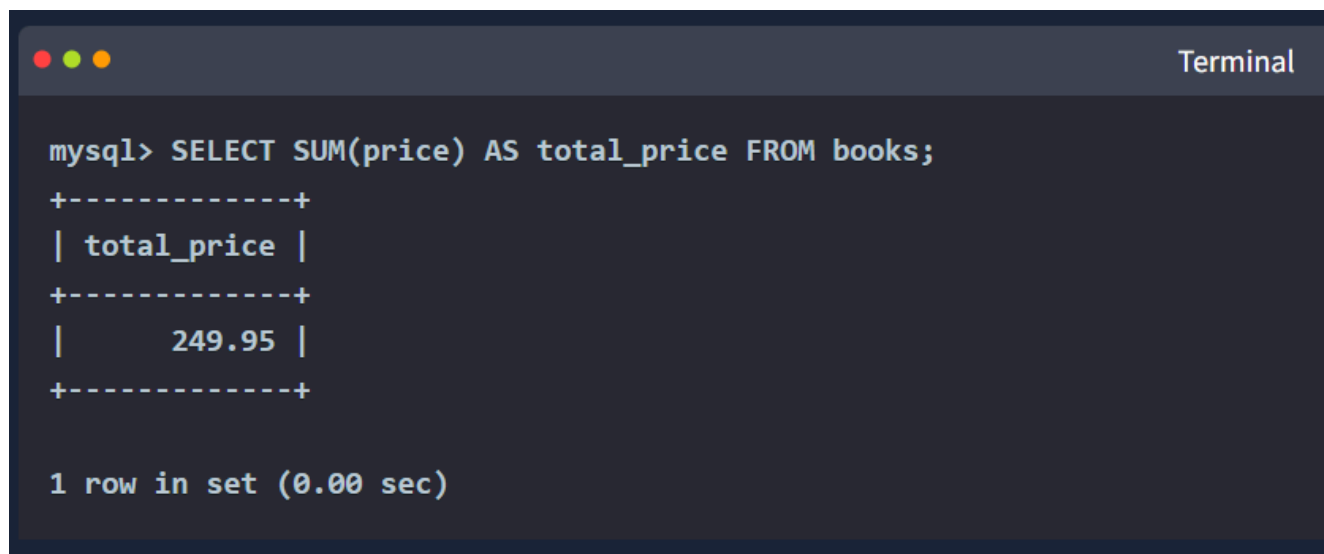
SUM() - Toplama

Belirli bir sütundaki sayısal değerlerin toplamını verir.

SQL

```
SELECT SUM(price) AS total_price FROM books;
```

- **Not:** Sadece sayısal (INT, FLOAT vb.) veriler içeren kolonlarda çalışır.



```
mysql> SELECT SUM(price) AS total_price FROM books;
+-----+
| total_price |
+-----+
|      249.95 |
+-----+

1 row in set (0.00 sec)
```

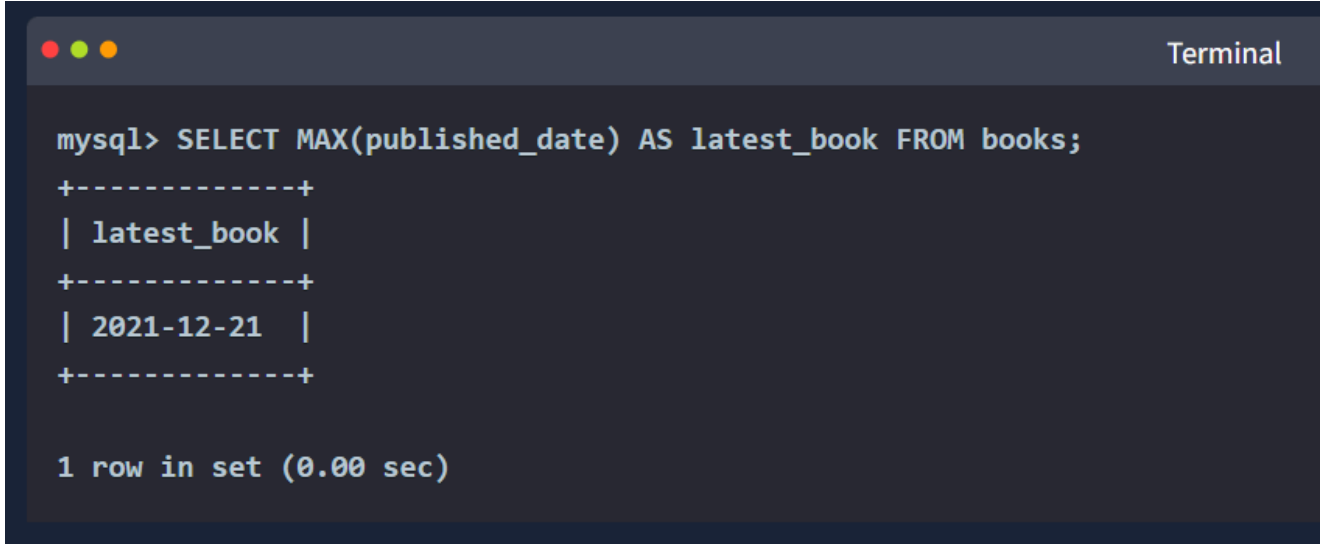
MAX() ve MIN() - En Büyük ve En Küçük

Belirtilen sütundaki en yüksek ve en düşük değerleri bulur.

- **MAX:** En yeni tarih, en yüksek fiyat veya alfabetik olarak en sondaki ismi getirir.
- **MIN:** En eski tarih, en düşük fiyat veya alfabetik olarak ilk ismi getirir.

SQL

```
SELECT MAX(published_date) AS latest_book FROM books; -- En yeni kitap
SELECT MIN(published_date) AS earliest_book FROM books; -- En eski kitap
```



```
mysql> SELECT MAX(published_date) AS latest_book FROM books;
+-----+
| latest_book |
+-----+
| 2021-12-21  |
+-----+

1 row in set (0.00 sec)
```

Özet Fonksiyon Tablosu

Fonksiyon	Türü	Ne Yapar?
CONCAT	String	Metinleri yan yana ekler.
SUBSTRING	String	Metnin içinden bir parça koparır.
LENGTH	String	Metin kaç karakterden oluşuyor?
COUNT	Aggregate	Kaç tane kayıt var?
SUM	Aggregate	Sayıların toplamı nedir?
MAX / MIN	Aggregate	En büyük / en küçük değer hangisi?

Bu fonksiyonlar özellikle **Blind SQL Injection** ve **Error-Based SQL Injection** gibi ileri seviye saldırılarda hayat kurtarır. Örneğin, veritabanı ismini doğrudan göremiyorsan, `LENGTH(database())` ile isminin kaç karakter olduğunu bulup sonra `SUBSTRING` ile harf harf tahmin yürütebilirsin.